

Programación Orientada a Objeto Seguro

Unidad Integradora de la competencia

Desarrollo de Software para una Agencia de Viajes
Caso: Viajes Aventura



Nombres:

Integrante 1 (Líder de Proyecto)

Integrante 2

Integrante 3, si corresponde

Docente: Prof. Rubén Schnettler

Asignatura: Programación Orientada a Objeto Seguro (TI3V21)

Carrera: Analista Programador

Institución: INACAP Valparaíso

Fecha: Octubre 2026

Índice

1. Introducción

2. Descripción del problema

- 2.1 Contexto de Viajes Aventura
- 2.2 Problemas actuales
- 2.3 Necesidad de una solución informática
- 2.4 Alcance y exclusiones

3. Levantamiento de requerimientos

- 3.1 Requerimientos funcionales
- 3.2 Requerimientos no funcionales
- 3.3 Reglas de negocio y trazabilidad
- 3.4 Priorización

4. Casos de uso

- 4.1 Actores
- 4.2 Diagrama de casos de uso
- 4.3 Fichas de casos de uso

5. Modelo UML

- 5.1 Diagrama de clases
- 5.2 Relaciones y decisiones de diseño

6. Modelo BPMN

- 6.1 Reserva de un paquete
- 6.2 Armado y publicación de un paquete
- 6.3 Eliminar o retirar un destino

7. Metodología ágil

- 7.1 Roles
- 7.2 Product Backlog
- 7.3 Sprint Backlog
- 7.4 Sprint
- 7.5 Criterios de terminado

8. Implementación

- 8.1 Python
- 8.2 Programación Orientada a Objetos
- 8.3 SQLite
- 8.4 CRUD
- 8.5 Persistencia

9. Seguridad

- 9.1 Autenticación
- 9.2 Contraseñas
- 9.3 Validaciones
- 9.4 Datos sensibles

10. Uso de Inteligencia Artificial

- 10.1 Prompts y recomendaciones
- 10.2 Análisis de las respuestas
- 10.3 Cambios realizados
- 10.4 Validación
- 10.5 Justificación del uso de IA

11. Pruebas

12. Conclusiones

13. Referencias

1. Introducción

Este informe documenta el desarrollo de un sistema de gestión para **Viajes Aventura**, agencia de Valparaíso que arma y vende paquetes turísticos combinando destinos propios. Hoy la agencia trabaja con una planilla, un cuaderno y mensajería, y de ese método nacen errores medibles: reservas duplicadas, cupos excedidos y precios cobrados distintos del publicado.

El documento recorre el trabajo en el orden en que se hizo: levantamiento de requerimientos, modelado (casos de uso, UML y BPMN), planificación ágil, implementación en Python con SQLite, seguridad, uso de inteligencia artificial y pruebas. Cada regla del caso se puede rastrear hasta el archivo y el método del código que la aplica.

2. Descripción del problema

2.1 Contexto de Viajes Aventura

Viajes Aventura fue creada hace dos años por tres socios. Planifica y vende paquetes propios, formados por destinos que ya recorrieron y cotizaron. El catálogo se lleva en una planilla, las reservas en un cuaderno y la atención por mensajería. Hoy registra **18 destinos** (4 ya no operan) y **214 reservas** en la última temporada.

2.2 Problemas actuales

Problema de la temporada	Cantidad	Cómo lo resuelve el sistema
Reservas duplicadas	19	Correo y RUT únicos; cada reserva se emite con su propio identificador
Reservas sobre el cupo	6	El cupo disponible se calcula con las personas ya reservadas antes de aceptar
Reservas sobre salidas vencidas	3	La fecha de salida se compara con la del día antes de reservar
Paquetes cobrados a otro precio	4	El precio se guarda al publicar el paquete y no se recalcula
Consultas resueltas revisando el cuaderno	31	El historial de cada cliente se obtiene con una consulta
Destinos no operados aún visibles	4	El destino se marca como no disponible, sin borrarlo

Además, los datos personales de los clientes están repartidos entre cuadernos, planillas y teléfonos personales, lo que expone información sensible.

2.3 Necesidad de una solución informática

La agencia necesita centralizar destinos, paquetes y reservas, reducir los errores de registro, controlar la disponibilidad y proteger los datos de sus clientes. Esto exige autenticación, validación de lo que se ingresa y almacenamiento seguro de credenciales.

2.4 Alcance y exclusiones

Dentro del alcance	Fuera del alcance
Destinos: registrar, modificar, retirar, eliminar y listar	Pasarela de pago y verificación de transferencias
Paquetes: combinar destinos, definir fechas, cupo y margen, calcular precio	Facturación electrónica ante el SII
Reservas: registro y acceso de clientes, reservar, consultar historial propio	Aplicación para teléfonos
Seguridad: autenticación por rol, validación de entradas, protección de credenciales y datos sensibles	Integración con aerolíneas, hoteles u operadores; envío de correos o mensajes; informes de gestión; contabilidad

3. Levantamiento de requerimientos

3.1 Requerimientos funcionales

Priorizados con MoSCoW: **Debe** (esencial), **Debería** (importante, no bloqueante) y **Podría** (queda para una versión posterior).

ID	Requerimiento funcional	Prioridad	Regla
RF-01	Registrar un destino con nombre, zona, descripción, duración y costo base	Debe	R1, R2
RF-02	Modificar los datos y el costo de un destino	Debe	R2
RF-03	Listar los destinos, distinguiendo los disponibles de los retirados	Debe	R8
RF-04	Eliminar un destino que no pertenece a ningún paquete	Debe	R8
RF-05	Retirar de la oferta un destino en uso (queda no disponible y se conserva)	Debe	R8
RF-06	Crear un paquete combinando de 2 a 5 destinos distintos	Debe	R3, R4
RF-07	Definir nombre, fechas de salida y regreso, cupo máximo y margen del paquete	Debe	R5, R6
RF-08	Calcular el precio por persona a partir de los costos de los destinos y el margen	Debe	R6
RF-09	Fijar el precio al publicar el paquete, sin cambios posteriores	Debe	R7
RF-10	Consultar los paquetes vigentes con precio, fechas y cupo disponible	Debe	R14, R15
RF-11	Registrar un cliente con nombre, RUT, correo único, teléfono y contraseña	Debe	R9
RF-12	Iniciar sesión con correo y contraseña (3 intentos)	Debe	R10, R11
RF-13	Reservar un paquete indicando la cantidad de personas (mínimo 1)	Debe	R11, R12, R16
RF-14	Rechazar una reserva que supere el cupo disponible	Debe	R14
RF-15	Rechazar una reserva sobre un paquete con salida vencida	Debe	R15
RF-16	Calcular el total de la reserva y conservarlo sin cambios	Debe	R13
RF-17	Consultar el historial de las reservas propias	Debe	R11
RF-18	Mostrar a cada rol solo las opciones que tiene permitidas	Debe	R11
RF-19	Cambiar la propia contraseña	Debería	R10
RF-20	Eliminar un paquete que no tiene reservas	Debería	R12
RF-21	Anular una reserva y liberar su cupo	Podría	—

3.2 Requerimientos no funcionales

ID	Requerimiento no funcional	Prioridad
RNF-01	Las contraseñas se almacenan con hash scrypt y salt aleatorio; nunca en texto plano	Debe
RNF-02	El acceso exige sesión y rol; cada cliente solo ve sus propias reservas	Debe
RNF-03	Toda entrada se valida en las clases de dominio, no solo en el menú	Debe
RNF-04	Todas las consultas SQL son parametrizadas (sin concatenar datos)	Debe
RNF-05	El RUT y el teléfono no aparecen en listados ni en mensajes de error	Debe
RNF-06	Los datos persisten en SQLite con restricciones NOT NULL, UNIQUE, PRIMARY KEY y FOREIGN KEY	Debe
RNF-07	Los errores se manejan con try/except/finally y rollback; el usuario recibe un mensaje claro, sin detalle técnico	Debe
RNF-08	Las clases de dominio no contienen SQL; el sistema usa solo la biblioteca estándar y corre en Windows, macOS y Linux	Debe

3.3 Reglas de negocio y trazabilidad

Las 17 reglas provienen del caso. La última columna indica dónde vive cada una en el código.

Regla	Enunciado	RF	Dónde se implementa
R1	Cada destino tiene nombre, zona, descripción, duración y costo base; el nombre es único	RF-01	models/destino.py · UNIQUE en destino
R2	El costo base es mayor que cero	RF-01, 02	Destino.__validar()
R3	Un paquete tiene de 2 a 5 destinos, sin repetir	RF-06	Paquete.__validar_destinos() · PK compuesta
R4	Un destino puede estar en varios paquetes	RF-06	Tabla paquete_destino
R5	Fechas y cupo; el regreso es posterior a la salida; cupo mayor que cero	RF-07	Paquete.__validar()

Regla	Enunciado	RF	Dónde se implementa
R6	Precio = suma de costos $\times$ (1 + margen); el margen no es negativo	RF-07, 08	Paquete.__calcular_precio()
R7	El precio queda fijo al publicar	RF-09	Columna precio_por_persona almacenada
R8	Destino libre se elimina; destino en uso se marca no disponible	RF-04, 05	DestinoService.eliminar_destino() / retirar_destino()
R9	El cliente se registra con sus datos; el correo es único	RF-11	Cliente.__validar() · UNIQUE en cliente
R10	La contraseña no se guarda en su forma original	RF-11, 12	seguridad.generar_hash()
R11	Solo un cliente autenticado reserva y consulta lo suyo	RF-12, 13, 17	Menú por rol · ReservaService.listar_por_cliente()
R12	Toda reserva registra cliente, paquete, fecha, personas y total	RF-13	Constructor de Reserva
R13	Total = precio $\times$ personas, sin cambios posteriores	RF-16	Paquete.calcular_total() · columna total
R14	No se acepta una reserva sobre el cupo disponible	RF-14	ReservaService.reservar() · SUM(cantidad_personas)
R15	No se reserva con la salida vencida	RF-15	Paquete.salida_vencida()
R16	Al menos una persona por reserva	RF-13	Reserva.__validar()
R17	RUT y teléfono no se muestran en listados ni errores	RNF-05	Cliente.__str__() los omite

3.4 Priorización

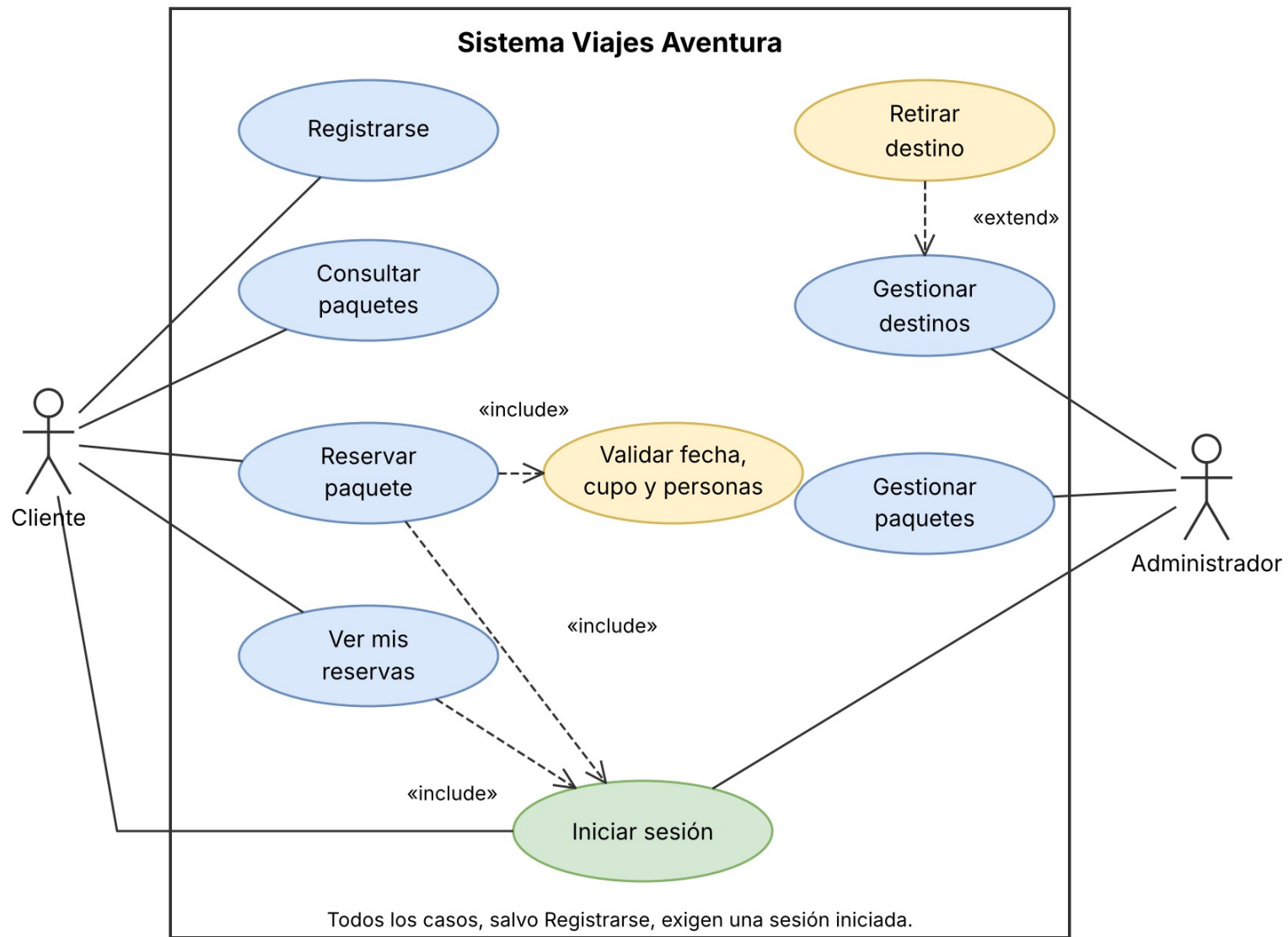
Los 18 requerimientos **Debe** (RF-01 a RF-18) forman el producto mínimo: sin ellos el caso no se resuelve. RF-19 y RF-20 mejoran la operación pero no bloquean el flujo. **RF-21 (anular reservas)** queda fuera de esta versión: anular libera cupo y obliga a definir devoluciones, algo que el caso no especifica.

4. Casos de uso

4.1 Actores

- **Cliente:** se registra, inicia sesión, consulta paquetes, reserva y revisa sus reservas.
- **Administrador:** socio de la agencia que mantiene el catálogo de destinos y los paquetes. Es un rol de la misma cuenta de persona, no una clase aparte.

4.2 Diagrama de casos de uso



Registrarse es el único caso sin sesión. «include» indica que el caso base siempre ejecuta al incluido; «extend», que el caso extensor ocurre solo en una condición (aquí, cuando el destino está en uso).

4.3 Fichas de casos de uso

CU • Reservar paquete

Campo	Contenido
Actor principal	Cliente
Requerimientos	RF-10, RF-13, RF-14, RF-15, RF-16
Precondición	El cliente inició sesión
Postcondición	La reserva queda almacenada con fecha, personas y total fijo
Flujo principal	1. El sistema muestra los paquetes vigentes con precio y cupo disponible. 2. El cliente elige un paquete e indica las personas. 3. El sistema valida salida vigente, personas ≥ 1 y cupo suficiente. 4. Calcula el total (precio $\times$ personas). 5. Registra la reserva y muestra el comprobante.
Flujos alternativos	3a. Salida vencida: informa y vuelve al listado. 3b. Personas sobre el cupo: informa cuántos cupos quedan. 3c. Personas < 1 : rechaza e indica la regla.

CU • Gestionar paquetes (armar y publicar)

Campo	Contenido
Actor principal	Administrador
Requerimientos	RF-06, RF-07, RF-08, RF-09
Precondición	Sesión de administrador iniciada; existen al menos 2 destinos disponibles
Postcondición	El paquete queda publicado con su precio fijo
Flujo principal	1. El administrador elige de 2 a 5 destinos. 2. Indica nombre, fechas, cupo y margen. 3. El sistema valida las reglas R3, R5 y R6. 4. Calcula el precio por persona. 5. Guarda el paquete y sus destinos en una sola transacción.
Flujos alternativos	3a. Datos inválidos (destino repetido, regreso anterior a la salida, margen negativo): el sistema informa la regla y vuelve a pedir los datos.

CU • Retirar destino

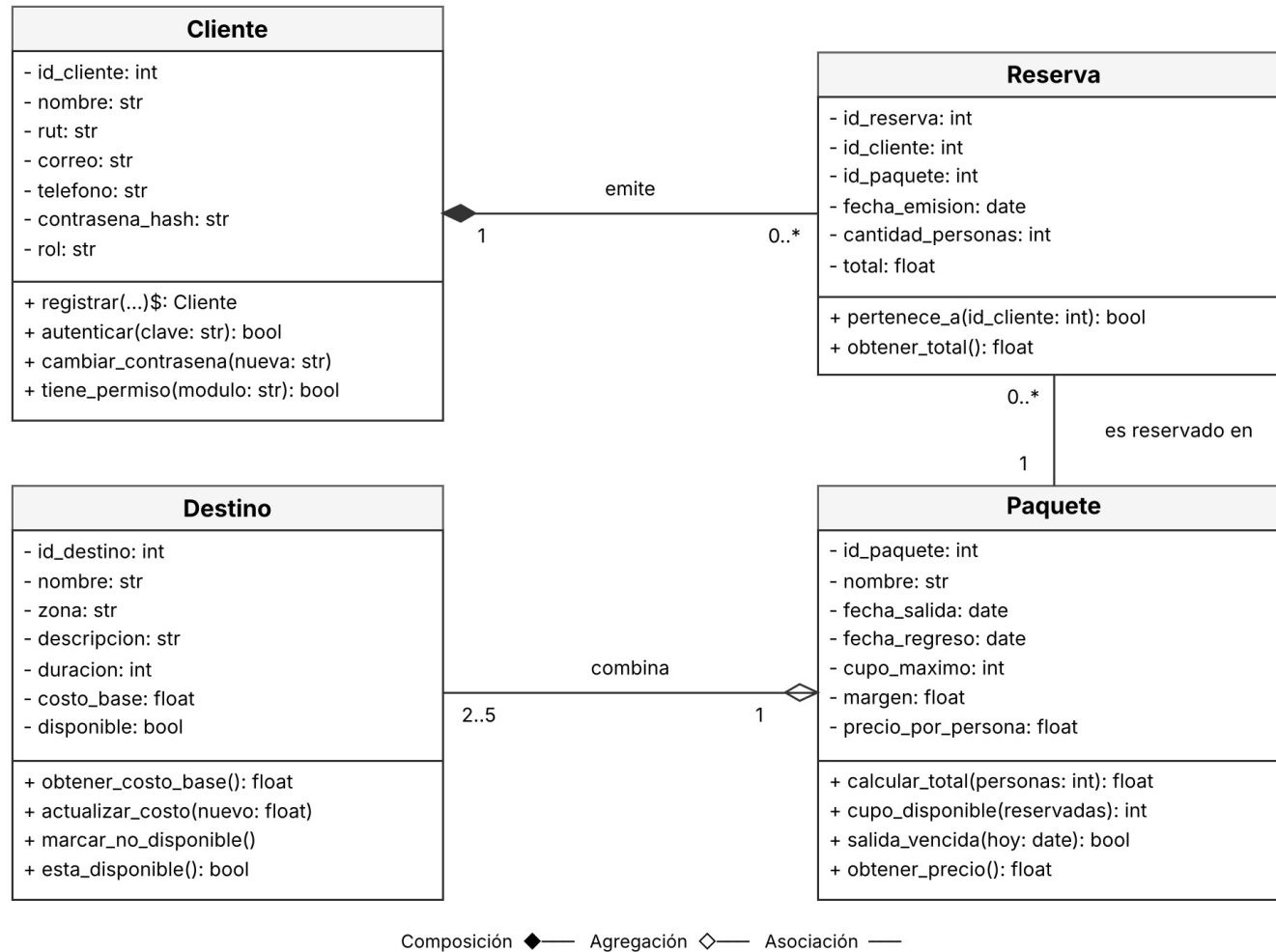
Campo	Contenido
Actor principal	Administrador
Requerimientos	RF-04, RF-05
Precondición	Sesión de administrador iniciada
Postcondición	El destino se elimina, o queda no disponible si estaba en un paquete
Flujo principal	1. El administrador selecciona el destino. 2. El sistema cuenta en cuántos paquetes participa. 3. Si en ninguno, lo elimina.
Flujos alternativos	3a. Si está en uso, rechaza la eliminación y sugiere retirarlo; al confirmar, lo marca no disponible y lo conserva dentro de los paquetes ya vendidos.

CU • Iniciar sesión

Campo	Contenido
Actor principal	Cliente o Administrador
Requerimientos	RF-12, RF-18, RNF-01
Precondición	La persona está registrada
Postcondición	La sesión queda abierta con el menú del rol
Flujo principal	1. El sistema pide correo y contraseña (esta no se muestra). 2. Valida las credenciales contra el hash. 3. Abre el menú de su rol.
Flujos alternativos	2a. Credenciales incorrectas: mensaje genérico, sin indicar cuál dato falló. Tras 3 intentos fallidos el sistema se cierra.

5. Modelo UML

5.1 Diagrama de clases



5.2 Relaciones y decisiones de diseño

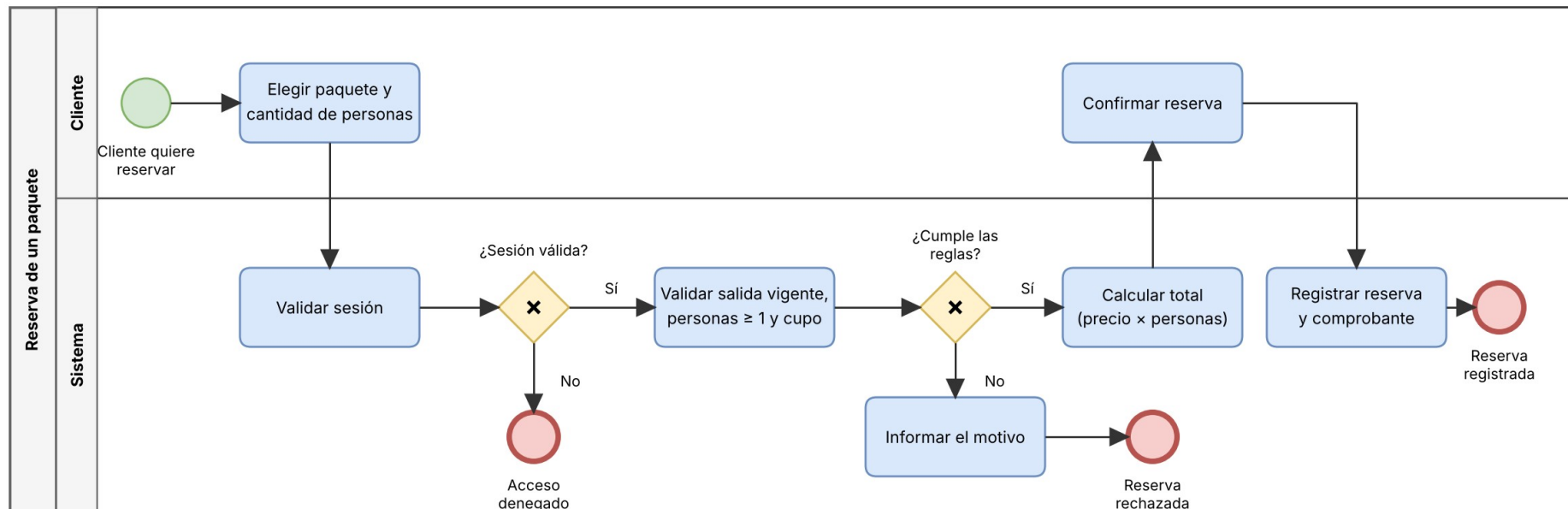
Relación	Tipo	Multiplicidad	Fundamento
Paquete — Destino	Agregación	2..5 a 0..*	R3 fija el rango y R4 permite un destino en varios paquetes. El destino existe sin el paquete
Cliente — Reserva	Composición	1 a 0..*	Una reserva solo tiene sentido con su cliente; por eso ON DELETE CASCADE
Paquete — Reserva	Asociación	1 a 0..*	El paquete existe sin reservas, y no se elimina mientras las tenga

- **El administrador es un rol, no una subclase.** Un socio puede dejar de administrar sin dejar de ser una persona registrada; con herencia habría que destruir el objeto y crear otro. El permiso se resuelve con `tiene_permiso()`.
- **El precio es atributo y el cupo disponible es método.** El precio debe quedar fijo (R7), así que se guarda. El cupo cambia con cada reserva, así que se calcula al consultarlo.
- **Las claves `id_*` y el margen están en el diagrama** porque el código los usa: sin ids no hay claves primarias ni foráneas, y sin margen no se puede calcular el precio (R6).
- **No hay método `anular()`** porque anular reservas quedó fuera del alcance (RF-21).

6. Modelo BPMN

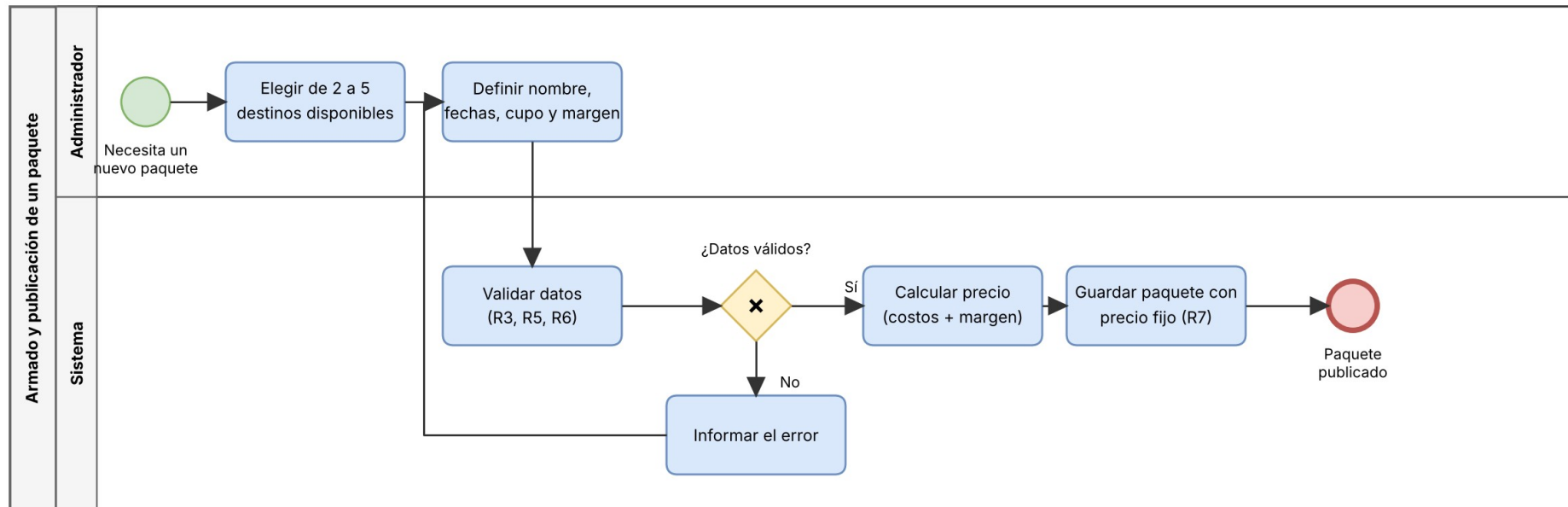
Los carriles separan a los participantes. Círculo verde: inicio; círculo rojo grueso: fin; rectángulo: tarea; rombo con x: compuerta exclusiva (se toma un solo camino).

6.1 Reserva de un paquete



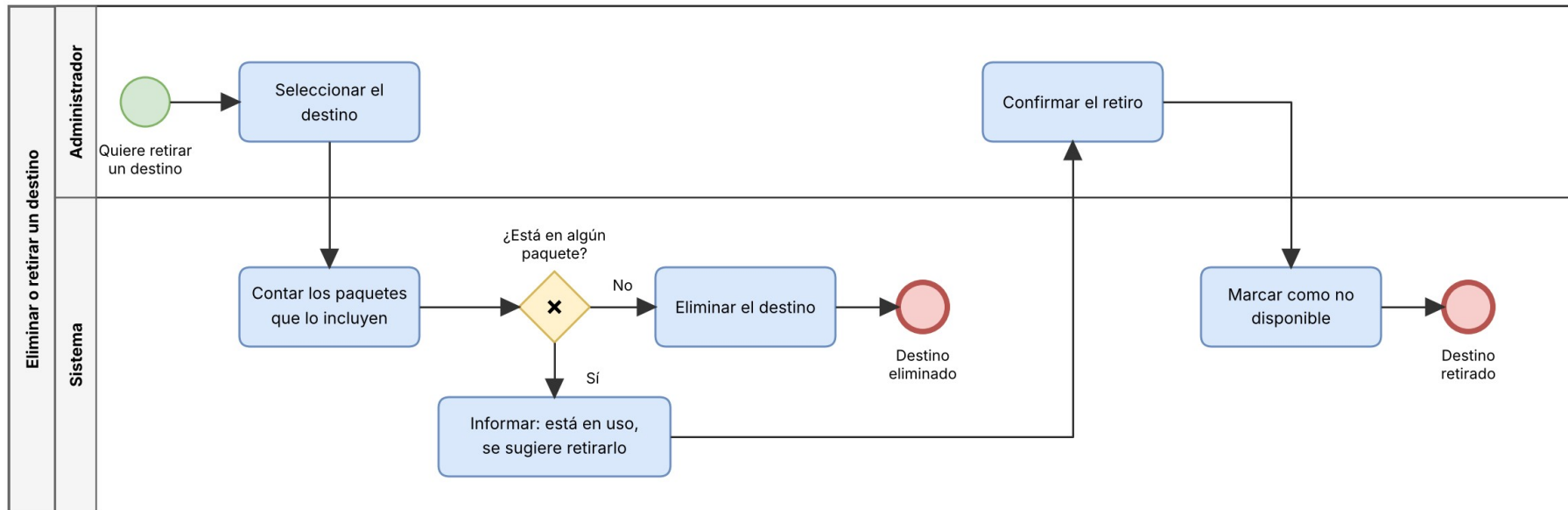
Lectura del proceso. El cliente elige el paquete y la cantidad de personas. El sistema valida la sesión y, si no es válida, deniega el acceso. Luego comprueba tres reglas juntas: salida vigente (R15), al menos una persona (R16) y cupo suficiente (R14). Si se cumplen, calcula el total con el precio fijo del paquete (R13), el cliente confirma y el sistema registra la reserva y emite el comprobante. Si alguna falla, informa el motivo y termina sin registrar nada.

6.2 Armado y publicación de un paquete



Lectura del proceso. El administrador elige de 2 a 5 destinos disponibles y define nombre, fechas, cupo y margen. El sistema valida las reglas R3, R5 y R6; si hay un error lo informa y el administrador corrige los datos. Con datos válidos calcula el precio por persona ($\text{costos} \times (1 + \text{margen})$) y lo guarda junto con el paquete en una sola transacción, de modo que el precio queda fijo (R7).

6.3 Eliminar o retirar un destino



Lectura del proceso. El sistema cuenta en cuántos paquetes participa el destino (R8). Si no está en ninguno, lo elimina. Si está en uso, informa que no puede eliminarse y sugiere retirarlo; al confirmar el administrador, el destino queda no disponible y se conserva dentro de los paquetes ya vendidos.

Los tres procesos son las operaciones donde el caso detectó errores: la reserva (cupó, fechas, total), el armado del paquete (precio) y el retiro del destino (conservar lo ya vendido). Los diagramas editables están en el archivo `diagramas_viajes_aventura.drawio`.

7. Metodología ágil

Se adoptó **Scrum con sprints de una semana**: con un equipo pequeño y cuatro semanas de plazo, una iteración más larga dejaría pocas oportunidades de corregir el rumbo.

7.1 Roles

Rol	Integrante	Responsabilidad
Product Owner	Los socios de la agencia (según el caso)	Fijan las necesidades y la prioridad de las reglas
Líder de Proyecto (rota)	Nombre	Coordina, hace seguimiento de las tareas y se comunica con el docente
Desarrollo · dominio y datos	Nombre	Clases de dominio, repositorios y esquema de la base de datos
Desarrollo · seguridad e interfaz	Nombre	Autenticación, permisos por rol, validación de entradas y menú

7.2 Product Backlog

ID	Historia de usuario	RF	Prioridad	Horas
HU-01	Como administrador quiero registrar, modificar y listar destinos para tener un catálogo único	RF-01 a 03	Debe	4
HU-02	Como administrador quiero retirar o eliminar destinos sin perder los paquetes ya vendidos	RF-04, 05	Debe	3
HU-03	Como administrador quiero crear y eliminar paquetes combinando destinos	RF-06, 07, 20	Debe	6
HU-04	Como administrador quiero que el precio se calcule y no cambie después	RF-08, 09	Debe	3
HU-05	Como cliente quiero registrarme e iniciar sesión de forma segura	RF-11, 12	Debe	6
HU-06	Como cliente quiero ver los paquetes vigentes con su cupo	RF-10	Debe	3
HU-07	Como cliente quiero reservar y conocer el total, sin pasar el cupo ni la fecha	RF-13 a 16	Debe	6
HU-08	Como cliente quiero consultar mi historial sin que otros lo vean	RF-17	Debe	3
HU-09	Como usuario quiero ver solo las opciones de mi rol	RF-18	Debe	3
HU-10	Como usuario quiero cambiar mi contraseña	RF-19	Debería	2
HU-11	Como cliente quiero anular una reserva y liberar el cupo	RF-21	Podría	5

Las estimaciones son horas de trabajo por historia. Las 9 historias **Debe** suman 37 horas; HU-10 agrega 2 y HU-11 no se incluye en esta versión.

7.3 Sprint Backlog

Sprint	Tarea	Responsable	Horas
1 · Sem. 15 Análisis y modelado	Levantar requerimientos y reglas de negocio	Todos	4
	Casos de uso y sus fichas	Líder	3
	Diagrama de clases	Datos	3
	Procesos BPMN	Seguridad	3
2 · Sem. 16 Dominio y persistencia	Esquema de la base de datos con restricciones	Datos	3
	Clases Destino y Paquete con validación	Datos	4
	Repositorios de destino y paquete	Datos	4
	Verificar que el precio publicado no cambia	Líder	1
3 · Sem. 17 Seguridad y reservas	Hash scrypt y autenticación	Seguridad	3
	Registro de clientes y permisos por rol	Seguridad	3

Sprint	Tarea	Responsable	Horas
	Reservas con sus validaciones	Datos	5
	Menú por rol y lectura segura de entradas	Líder	3
4 · Sem. 18 Integración	Pruebas de las 17 reglas (pruebas.py)	Todos	3
	Revisión crítica del código con apoyo de IA	Todos	2
	Informe técnico	Líder	4
	Preparación de la defensa individual	Todos	2

7.4 Sprint

Cada sprint dura una semana, empieza eligiendo tareas del Product Backlog y termina con una funcionalidad ejecutable y probada. Los objetivos fueron: **Sprint 1**, dejar el análisis y los modelos; **Sprint 2**, la base de datos y la gestión de destinos y paquetes; **Sprint 3**, seguridad y reservas; **Sprint 4**, pruebas, revisión del código e informe.

7.5 Criterios de terminado

Criterio	Descripción
Ejecutable	La funcionalidad corre desde el menú del sistema sin errores
Regla probada	La regla asociada se probó con un caso que debe fallar y efectivamente falla
Validación en el dominio	La validación vive en la clase, no solo en el menú
Consultas seguras	Las operaciones con la base usan consultas parametrizadas
Revisión cruzada	Otro integrante probó la funcionalidad
Repositorio	El código probado se sube al repositorio con un commit

8. Implementación

8.1 Python

El sistema usa **Python 3** y solo su biblioteca estándar (sqlite3, hashlib, hmac, os, datetime, getpass). No hay dependencias que instalar, por lo que corre igual en Windows, macOS y Linux.

8.2 Programación Orientada a Objetos

- **Encapsulación:** los atributos son privados (`__nombre`) y se leen con métodos `obtener_*`.
- **Validación en el constructor:** un objeto inválido no llega a existir (por ejemplo, un Paquete con un solo destino).
- **Responsabilidad en quien sabe:** `Paquete.calcular_total()` y `Reserva.pertenece_a()` viven en sus clases, no en el menú.
- **Patrón repositorio:** `models/` dice qué es cada cosa; `services/` la guarda y la recupera. Ninguna clase de dominio contiene SQL.

Carpeta	Contenido
database/conexion.py	Conexión (con <code>PRAGMA foreign_keys = ON</code>) y creación de tablas
models/	<code>destino.py</code> , <code>paquete.py</code> , <code>cliente.py</code> , <code>reserva.py</code>
services/	Repositorios de las cuatro clases y <code>seguridad.py</code> (hash de contraseñas)
main.py, pruebas.py	Registro, login y menú por rol; pruebas de las reglas

8.3 SQLite

Tabla	Claves y restricciones
destino	PK <code>id_destino</code> · nombre UNIQUE · disponible (borrado lógico)
paquete	PK <code>id_paquete</code> · precio_por_persona almacenado (R7)
paquete_destino	PK compuesta (<code>id_paquete</code> , <code>id_destino</code>) · FK a ambas · cascada al borrar el paquete

Tabla	Claves y restricciones
cliente	PK id_cliente · correo y rut UNIQUE · rol
reserva	FK a cliente (ON DELETE CASCADE) y a paquete (sin cascada) · total almacenado

8.4 CRUD

Entidad	Crear	Leer	Actualizar	Eliminar
Destino	Registrar destino	Ver catálogo	Modificar datos / cambiar costo	Eliminar (si no está en uso) o retirar
Paquete	Crear paquete	Ver publicados / disponibles	— (el precio es fijo)	Eliminar (si no tiene reservas)
Cliente	Registrarse	Autenticar	Cambiar contraseña	—
Reserva	Reservar paquete	Ver mis reservas	— (el total es fijo)	— (anular queda fuera)

8.5 Persistencia

Cada operación abre su conexión, trabaja dentro de una transacción y la cierra en finally. Si algo falla se hace `rollback()` y el error se relanza con `raise ... from error`, de modo que el usuario ve un mensaje claro y el detalle técnico queda encadenado para depurar. Las excepciones se capturan de la más concreta (`IntegrityError`) a la más general (`sqlite3.Error`).

9. Seguridad

Aspecto	Cómo se implementa
9.1 Autenticación	Login con correo y contraseña, máximo 3 intentos. El mensaje de error es genérico: no revela si falló el correo o la contraseña. La contraseña se lee con <code>getpass</code> , sin mostrarla
Control de acceso	Sin sesión no se entra. <code>PERMISOS_POR_ROL</code> define los módulos de cada rol y el menú se arma solo con ellos: las opciones no permitidas no aparecen
Aislamiento entre clientes	Las consultas de reservas siempre filtran por el cliente de la sesión; aunque alguien adivine el id de otra reserva, <code>per_tenece_a()</code> la trata como inexistente
9.2 Contraseñas	<code>hashlib.scrypt</code> con salt aleatorio por contraseña; se compara con <code>hmac.compare_digest</code> (tiempo constante). Mínimo 8 caracteres combinando letras y números. Nunca se guarda el texto original
9.3 Validaciones	En los constructores de las clases de dominio, más conversión controlada de lo que se escribe en el menú (enteros, decimales y fechas). Normalización de correo y RUT
Inyección SQL	Todas las consultas usan parámetros <code>?</code> ; no hay concatenación ni f-strings con datos del usuario dentro del SQL
Restricciones en la base	NOT NULL, UNIQUE, PRIMARY KEY y FOREIGN KEY con <code>PRAGMA foreign_keys = ON</code> : la base también defiende las reglas
9.4 Datos sensibles	El RUT y el teléfono tienen métodos de acceso, pero ningún listado ni mensaje de error los usa; <code>Cliente.__str__()</code> los omite

10. Uso de Inteligencia Artificial

10.1 Prompts y recomendaciones

Se usó IA como apoyo para proponer una primera versión del código y para revisarla. Los prompts textuales que usó cada integrante son:

Integrante	Prompt utilizado (texto textual)	Para qué
Nombre	Pegar aquí el prompt tal como se escribió	Ej.: primera versión de las clases
Nombre	Pegar aquí el prompt tal como se escribió	Ej.: revisión de seguridad
Nombre	Pegar aquí el prompt tal como se escribió	Ej.: consultas SQL

10.2 Análisis de las respuestas y 10.3 Cambios realizados

La IA no se aceptó tal cual. Cada propuesta se contrastó con las reglas del caso y se modificó o descartó cuando las contradecía:

Aspecto	Problema detectado	Decisión	Fundamento técnico
Precio del paquete	Se calculaba en cada consulta sumando costos	Modificar	Contradice R7: un cambio de costo alteraría paquetes ya vendidos. Se guarda en la tabla
Eliminar destinos	DELETE directo	Modificar	R8 exige conservar el destino en uso. Se agregó disponible y una verificación previa
Hash de contraseñas	sha256 simple	Descartar	Sin salt es vulnerable a tablas precalculadas. Se usó script con salt
Comparar hashes	Operador ==	Modificar	Permite ataques por tiempo de respuesta. Se usó hmac.compare_digest
Cupo	Contaba reservas	Modificar	Una reserva de 4 personas ocupa 4 cupos. Se usó SUM(cantidad_personas)
Listado de clientes	Mostraba RUT y teléfono	Modificar	R17: son datos sensibles. __str__ los omite
Consultas SQL	Concatenaba los datos	Descartar	Habilita inyección SQL. Todo usa parámetros ?
Manejo de errores	Un solo except Exception	Modificar	Oculto la causa. Se separaron ValueError y sqlite3.Error
Validación	Solo en el menú	Modificar	Se salta llamando a la clase directamente. Se trasladó a los constructores

10.4 Validación

Cada cambio se verificó con pruebas.py (sección 11): cada caso intenta violar una regla y comprueba que el sistema la rechaza. Además, el código se leyó línea por línea contra la tabla de reglas de la sección 3.3.

10.5 Justificación del uso de IA

La IA acelera la escritura de código repetitivo, pero no conoce las reglas del negocio ni sus consecuencias. Por eso se usó como borrador y no como autoridad: las decisiones de diseño (precio fijo, borrado lógico, rol en vez de subclase) se tomaron y se pueden defender sin ella.

11. Pruebas

pruebas.py ejecuta 26 casos sobre una base temporal (no toca viajes.db). Resultado de la última ejecución: **26 de 26 correctos**. Para correrlas: py pruebas.py.

Regla	Caso probado	Esperado
R1, R2	Nombre de destino repetido; costo base igual a cero	Se rechaza
R3, R4	Paquete con 1 destino, con 6 y con uno repetido; un destino en dos paquetes	Se rechazan los 3 primeros; el último se acepta
R5, R6	Regreso anterior a la salida; margen negativo; precio = costos × (1 + margen)	Se rechazan los dos primeros; el precio coincide
R7	Subir el costo de un destino después de publicar el paquete	El precio publicado no cambia
R8	Eliminar un destino en uso; retirarlo; eliminar uno libre	Rechaza; retira; elimina
R9, R10	Correo repetido; contraseña sin números; hash en la base; login correcto e incorrecto	Rechaza; rechaza; no está en claro; autentica solo la correcta
R11	Un cliente intenta ver la reserva de otro	Se trata como inexistente
R13	Total de una reserva de 2 personas	Precio × 2
R14	Reservar sobre el cupo; reservar dentro del cupo	Rechaza; acepta
R15, R16	Reservar con salida vencida; reservar 0 personas	Se rechazan
R17	str(cliente) no contiene RUT ni teléfono	Correcto
Inyección SQL	Login con ' OR '1'='1	No autentica ni rompe

12. Conclusiones

El sistema resuelve los seis problemas medidos en el caso: las 17 reglas están implementadas, ubicadas en el código y verificadas con pruebas. Las decisiones que más pesan son tres: el **precio se guarda** (R7), el **destino en uso se retira en vez de borrarse** (R8) y el **administrador es un rol**, no una clase.

Quedan fuera de esta versión la anulación de reservas, el pago en línea y los informes de gestión. La arquitectura en capas (dominio, repositorios, menú) permite agregarlos sin tocar las reglas existentes. Como aprendizaje, el trabajo mostró que la IA propone rápido pero se equivoca en lo que depende del negocio, y que la validación y la seguridad deben vivir en el dominio y en la base, no en la pantalla.

13. Referencias

- INACAP. Caso Viajes Aventura y guía de la Evaluación Sumativa 4, TI3V21 Programación Orientada a Objeto Seguro (2026).
- Schnettler, R. Material de clases 1 a 6, Programación Orientada a Objeto Seguro, INACAP.
- Python Software Foundation. Documentación de la biblioteca estándar: `sqlite3`, `hashlib` (`scrypt`), `hmac`, `getpass`. <https://docs.python.org/3/>
- SQLite. Foreign Key Support. <https://www.sqlite.org/foreignkeys.html>
- Schwaber, K. y Sutherland, J. La Guía de Scrum (2020). <https://scrumguides.org/>
- Object Management Group. UML 2.5 y BPMN 2.0 (especificaciones).